



TECHNISCHE UNIVERSITÄT
CHEMNITZ

Term Paper

**Subject : Open Data Integration with
Accidents in Germany**

**Subject: Datenbanken und
Web-Techniken**

Presented To:

Prof. Dr. Michael Martin and Florian Hahn

Presented By:

Student Name: Meeta Pareshbhai Dave

Field of Study: Masters in Web Engineering

Student Id: 898021

Abstract

This project presents the design and implementation of a data-integration platform for official German road-accident statistics, developed as part of the module *Datenbanken und Web-Techniken* at Technische Universität Chemnitz. The platform integrates 2,098,019 accident records across nine annual releases (2016–2024) from the Unfallatlas open-data portal, regional statistics from GENESIS/Regionalstatistik, and district boundary geometries from the Regionalatlas, combining three independently structured sources into a single canonical database.

A central component of the platform is a column-variant resolution mechanism that harmonises the differing attribute names, file encodings, decimal conventions, and column orderings found across the nine annual Unfallatlas releases. For example, the road-condition attribute is named *IstStrasse* in 2016, *STRZUSTAND* in 2017–2020, and *IstStrassenzustand* from 2021 onwards. All three sources are joined on the official Amtlicher Gemeindegliederung (AGS), a hierarchical eight-digit region key that enables efficient prefix-based filtering across country, state, district, and municipality levels without requiring spatial joins.

The integrated dataset is exposed through a RESTful API built with FastAPI, which auto-generates OpenAPI/Swagger documentation and supports query categories including temporal aggregation, regional ranking, spatial hotspot detection, and multi-source rate computation such as accidents per 100,000 registered passenger cars. A React-based analytical dashboard consumes the API, providing interactive choropleth maps, severity-ranked hotspot visualisation, and year-over-year trend analysis. All API responses include structured provenance metadata with source attribution and licence information under Datenlizenz Deutschland – Namensnennung 2.0.

The platform is fully reproducible: a single import command rebuilds the entire database from raw source files, and automated data-quality checks confirm structural integrity across all loaded records, including coordinate validity, AGS format compliance, and duplicate detection.

Content

- 1. Introduction.....5
- 2. Data Collection..... 6
- 3. Data Analysis and Source Mapping.....7
- 4. Database Schema Design.....8
- 5. ETL Pipeline..... 9
- 6. API Design.....10
- 7. Frontend Dashboard..... 11
- 8. Data Quality and Evaluation..... 12
- 9. Conclusion..... 13

List of Illustrations

- Figure 3.1.15
- Figure 3.2.15
- Figure 3.3.16
- Figure 3.3.2.....7
- Figure 5.3.110
- Figure 11.3.114
- Figure 11.3.2.....14
- Figure 11.3.315
- Figure 11.3.4.....15

1. Introduction

This project develops a data-integration platform for official German road-accident statistics as part of the module *Datenbanken und Web-Techniken* at Technische Universität Chemnitz. The platform combines multiple open government datasets into a unified database and exposes the integrated data through a documented REST API and an interactive web dashboard.

The primary data source is the Unfallatlas, published annually by the German states via opengeodata.nrw.de, which records every road accident involving personal injury at the municipality level. This is supplemented by registered passenger-car statistics from the GENESIS regional database (table 46251-01-03-4) and district boundary geometries from the Regionalatlas. Together, these three sources enable queries that cannot be answered from any single dataset alone — for example, the number of accidents per 100,000 registered passenger cars in a given district.

The platform is structured in three layers: an ETL pipeline that extracts, cleans, and loads the raw source files into a canonical SQLite database; a RESTful API that serves the integrated data as structured JSON responses; and a React-based frontend dashboard that visualises the results. Every component is reproducible — the entire database can be rebuilt from the raw source files with a single command.

2. Data Collection

The platform integrates three official German open datasets. All sources are published under the Datenlizenz Deutschland – Namensnennung 2.0 (dl-de/by-2-0), which permits free use and redistribution with attribution. Table 1 provides an overview of each source.

Source	Publisher	URL	Format	Region Key	Content	Years
Unfallatlas	German states via opengeodata.nrw.de	https://www.opengeodata.nrw.de/produkte/transport_verkehr/unfallatlas/	CSV (semicolon-delimited)	AGS (8-digit)	One row per road accident with personal injury; severity, participant types, coordinates, time	2016–2024
GENESIS 46251-01-03-4	Statistisches Bundesamt / KBA	https://www.regionalstatistik.de/genesis/online (table 46251-01-03-4)	CSV (latin-1)	AGS (5-digit)	Registered passenger cars per district; reference date 01.01.	2024
Regionalatlas boundaries	Statistikportal / isellsoap	https://github.com/isellsoap/deutschlandGeoJSON/blob/main/4_kreise/4_nie	GeoJSON	RS (5-digit)	District boundary polygons for ~400 Kreise	—

Source	Publisher	URL	Format	Region Key	Content	Years
		drig.geo.json				
Regionalstatistik per-10k	Regionalstatistik.de	https://www.regionalstatistik.de/genesis/online	CSV (UTF-8)	AGS (5-digit)	Accidents per 10,000 inhabitants per district	2023

Table 1: Integrated data sources

The four data sources integrated into the platform are summarised in Table 1. All sources are joined on the official AGS region key and published under Datenlizenz Deutschland – Namensnennung 2.0 (dl-de/by-2-0), permitting free use and redistribution with attribution.

3. Data Analysis and Source Mapping

Before building the schema or writing any loader, all four source files were inspected to understand their structure, identify inconsistencies, and define the mapping to the canonical schema. This analysis revealed that the Unfallatlas files — despite representing the same dataset — differ structurally across annual releases in several ways simultaneously.

Column naming drift across years

The most significant finding was that key attribute names change from year to year. Table 2 documents the drift for the five most affected columns across all nine releases. This made a static column-mapping approach insufficient; a variant-resolution mechanism was required instead.

Canonical field	2016	2017	2018	2019	2020	2021	2022	2023	2024
Record ID	FID / OBJECTID	OBJECTID / UIDENTSTLA	OBJECTID_1	OBJECTID	OBJECTID / UIDENTSTLAE	OID_ / UIDENTSTLAE	OBJECTID / UIDENTSTLAE	OID_ / UIDENTSTLAE	OID_ / UIDENTSTLAE
Light conditions	ULICHTVERH	LICHT	ULICHTVERH	ULICHTVERH	ULICHTVERH	ULICHTVERH	ULICHTVERH	ULICHTVERH	ULICHTVERH
Road condition	IstStrasse	STRZUSTAND	STRZUSTAND	STRZUSTAND	STRZUSTAND	IstStrassenzustand	IstStrassenzustand	IstStrassenzustand	IstStrassenzustand
Other participant	IstSonstige	IstSonstige	IstSonstige	IstSonstige	IstSonstige	IstSonstige	IstSonstige	IstSonstige	IstSonstige
Goods vehicle	IstGkfst	(absent)	IstGkfst	IstGkfst	IstGkfst	IstGkfst	IstGkfst	IstGkfst	IstGkfst

Table 2: Unfallatlas column name drift across annual releases (2016–2024)

The table documents the attribute naming inconsistencies observed across the nine annual Unfallatlas releases. Four canonical fields change name at least once across the nine years, with the road-condition attribute cycling through three distinct spellings. This drift makes a static column-mapping approach insufficient and directly motivated the column-variant resolution mechanism implemented in the ETL pipeline.

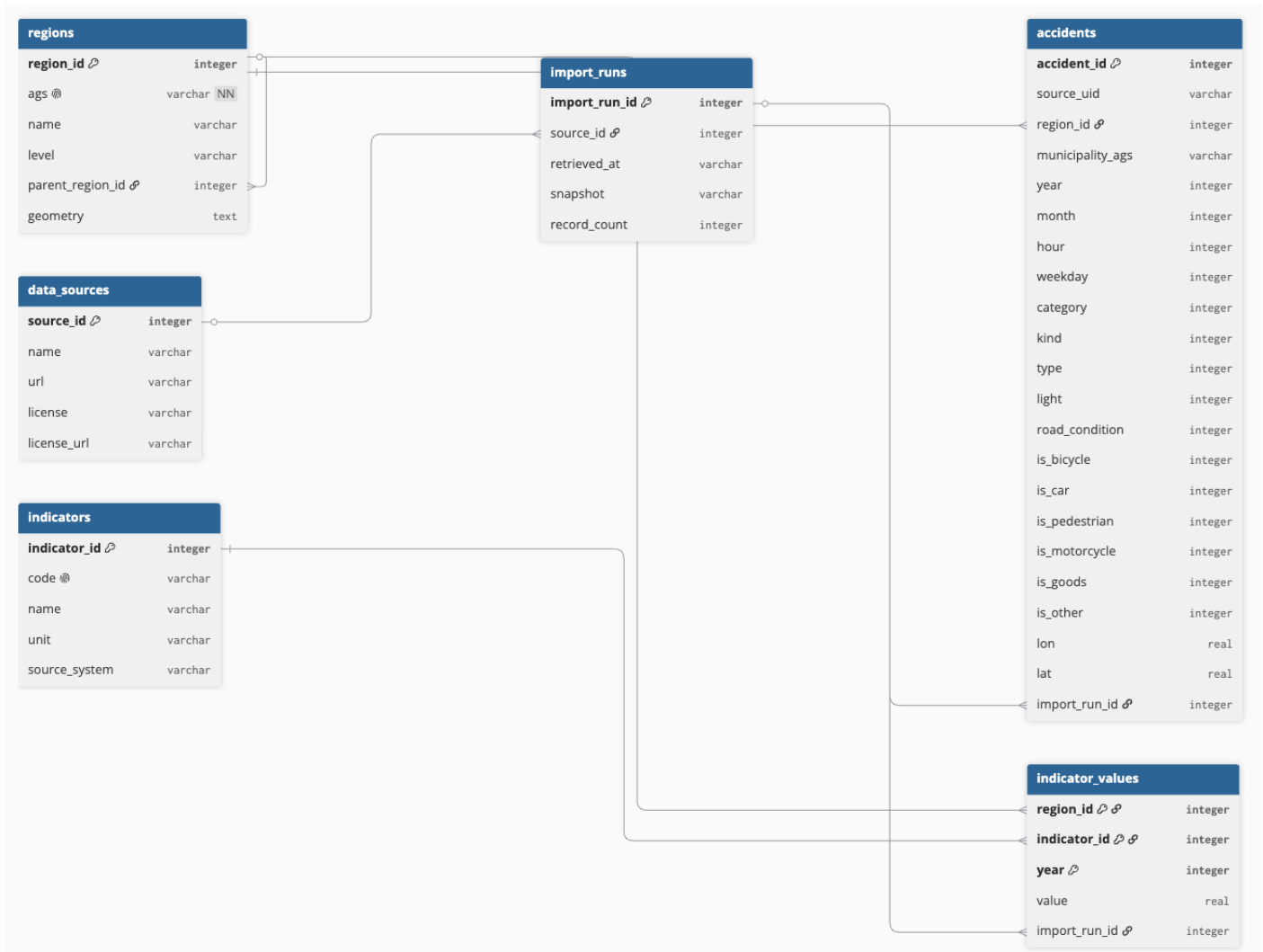
Additional structural differences

Beyond column naming, the following differences were observed across the nine releases and handled explicitly in the ETL pipeline:

- File encoding: releases from 2016–2020 use plain ASCII; releases from 2021 onwards use UTF-8 with a byte-order mark (BOM).
- Decimal separator: coordinates are stored with a comma as the decimal separator throughout all releases (e.g. 10,621659), requiring normalisation before numeric parsing.
- File extension: the 2016 release uses .txt; later releases use .csv.
- Folder nesting: the 2021 release is nested inside an additional subfolder (Unfallorte2021_EPSG25832_CSV/), unlike all other years.
- Missing columns: the IstGkfz (goods vehicle) column is entirely absent from the 2017 release and is defaulted to zero during loading.
- AGS composition: the region key is never stored as a single column — it is split across four fields (ULAND, UREGBEZ, UKREIS, UGEMEINDE) that must be zero-padded and concatenated to form the canonical 8-digit AGS.

4. Database Schema Design

The canonical database schema was designed based on the source analysis in the previous section. The primary design goal was to support all required query categories — temporal filtering, regional aggregation, multi-source rate computation, and spatial hotspot detection — while keeping the structure reproducible and source-independent. SQLite was chosen as the database engine for its zero-setup, file-based nature, which supports the reproducibility requirement: the entire database is rebuilt from raw files with a single command and can be distributed as a single file.



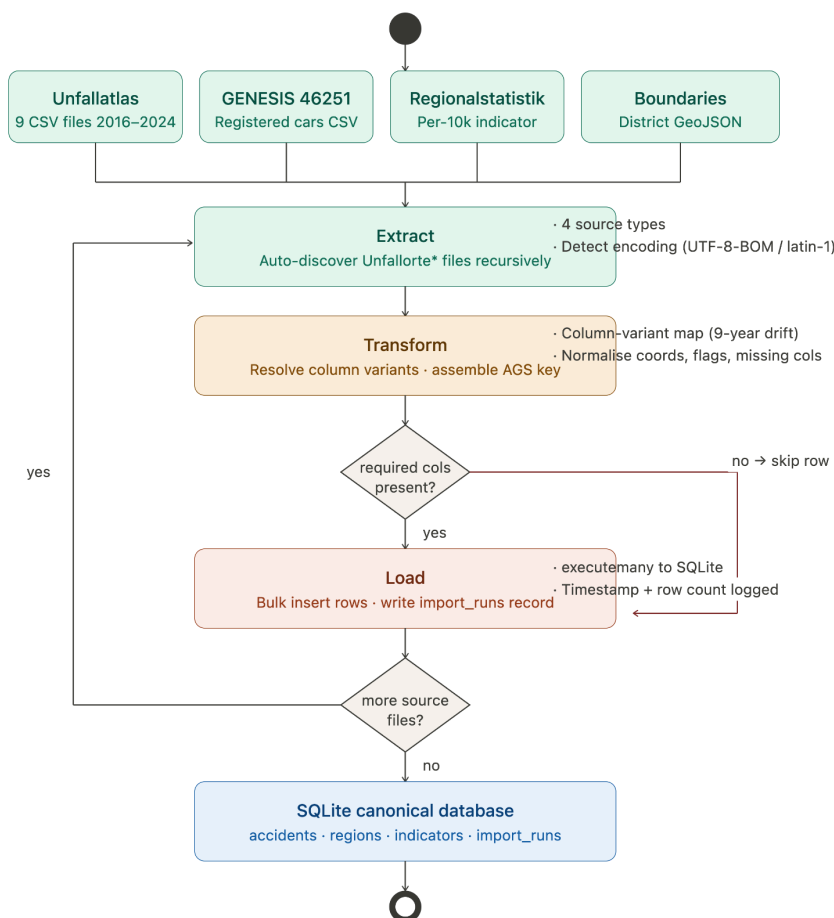
Entity-Relationship Diagram of the Canonical Database Schema

The schema consists of six tables centred around the **accidents** table, which stores one row per recorded accident and links to the **regions** table via the AGS region key. The **regions** table encodes the full administrative hierarchy — country, state, district, and municipality — through a self-referencing **parent_region_id** foreign key, and carries a **geometry** column for district polygon data used by the choropleth map. Time-varying regional statistics such as registered car counts and per-10k rates are stored in **indicator_values**, linked to named **indicators**, allowing any number of denominators to be added without

schema changes. The two provenance tables — `data_sources` and `import_runs` — record the origin, retrieval timestamp, and row count of every import, ensuring the lineage of every answer is traceable and returned to API clients in each response.

5. ETL Pipeline

The ETL (Extract, Transform, Load) pipeline is responsible for reading all four raw source files, harmonising their differing structures, and loading the cleaned data into the canonical SQLite database.



Extract — All four source files are read from `data/raw/`. Unfallatlas accident files are auto-discovered recursively regardless of folder depth or file extension, and encoding is detected automatically (UTF-8-BOM, cp1252, latin-1).

Transform — A column-variant map resolves the nine-year attribute-name drift by matching each file's actual header to a canonical field list at load time. The 8-digit AGS key is assembled from four source columns (ULAND + UREGBEZ + UKREIS + UGEMEINDE), coordinates are normalised from comma to dot decimals, and participant flags are coerced to integer. Missing columns (e.g. `IstGkfz` absent in 2017) default to zero.

Load — Cleaned rows are bulk-inserted using `executemany`. The database is fully reset at the start of each run to prevent stale data. Every load is recorded in `import_runs` with a timestamp, source reference, and row count, forming a complete audit trail.

Loader	Source	Records loaded	Key transformation
load_accidents.py	Unfallatlas 2016–2024 (9 files)	2,098,019	Column-variant resolution, AGS assembly, coordinate normalisation
load_indicators.py	GENESIS 46251 (cars)	~400 districts	Multi-header skip, column index parsing, integer cleaning
load_indicators.py	Regionalstatistik per-10k	~400 districts	Flat-table parsing, dot decimal normalisation
load_geometries.py	District GeoJSON	~400 polygons	RS→AGS prefix match, GeoJSON stored as text
load_regions.py	Fixed reference data	16 states + hierarchy	AGS prefix decomposition, parent-child linking

ETL Loader Summary

The platform's ETL pipeline is implemented across four dedicated loader modules, each responsible for a specific source type. Together they load 2,098,019 accident records, registered car counts and per-10k rates for approximately 400 districts, district boundary polygons, and the 16-state regional hierarchy — all into the canonical schema in a single reproducible run.

6. API Design

The platform exposes the integrated database through a RESTful API built with FastAPI, comprising 15 endpoints that cover five query categories: counts, rankings, spatial queries, trend analysis, and multi-source rates. Every endpoint uses HTTP GET, returns structured JSON, and includes a `provenance` block carrying the data source attribution and licence with every response. FastAPI auto-generates a full OpenAPI/Swagger documentation page at `/docs` directly from the code, ensuring documentation stays in sync with the implementation at all times. Errors follow a consistent shape across all endpoints, an in-memory cache with a 300-second TTL is applied to all responses, and state filters accept both human-readable codes (e.g. `SN`, `BE`) and 2-digit AGS prefixes.

Endpoint	Query category	Key parameters
<code>/aggregates/accidents</code>	Count	<code>state</code> , <code>year</code> , <code>category</code> , participant flags

/aggregates/accidents/by-region	Ranking	level, year, order, limit
/aggregates/rate	Multi-source rate	indicator, level, year, per
/aggregates/hotspots	Spatial clustering	state, year, precision, weighted
/accidents/near	Spatial radius	lat, lon, radius_km, year
/regions/choropleth	GeoJSON choropleth	metric, year
/stats/first-year	Temporal	state
/stats/trend	Trend series	state, category
/metadata/sources	Provenance	—

Key API endpoints

The Table lists the primary endpoints of the platform. Each endpoint addresses a single query category, keeping the interface predictable and independently extensible. The two multi-source endpoints — /aggregates/rate and /aggregates/hotspots — are the most analytically significant, as they combine accident records with regional indicators and spatial clustering respectively, answering questions that cannot be resolved from any single source alone.

7. Frontend Dashboard

- Built with React + Vite + TypeScript, styled with Tailwind CSS — a production-grade stack with no backend coupling; the frontend consumes the API exclusively and never accesses the database directly.
- A shared filter bar (region, year, severity) drives all panels simultaneously — every component always reflects the same query context.
- Headline KPI panel displays the total accident count for the selected filter with a fatal / serious / light severity split rendered as proportional bars.
- Hotspot map (Leaflet) plots severity-ranked crash clusters across the selected region — markers are sized by frequency and colored by harm (red = fatal, amber = serious, blue = light).
- Choropleth map (Leaflet) shades Germany's ~400 districts by accident count using a five-class quantile color scale — powered by the /regions/choropleth GeoJSON endpoint.

- Ranking chart (Recharts) shows the top 10 districts by accident count for the selected filter, with exact values labelled on each bar.
- Trend line (Recharts) plots year-over-year accident counts across all nine loaded years (2016–2024), with the currently selected year highlighted.
- All data is fetched live from the API on every filter change; every panel displays the provenance and licence information returned with each response.

8. Data Quality and Evaluation

- A dedicated quality-check script (`quality_checks.py`) runs 15 automated checks immediately after import and exits non-zero on any failure, making it suitable as a gate in a CI/update workflow.
- All 15 checks pass on the full 2,098,019-row dataset — zero out-of-range coordinates, zero invalid AGS keys, zero bad category values, zero duplicate source identifiers, and zero unlinked accident rows.
- Coordinate validity — all longitude and latitude values fall within Germany's bounding box (5.5°–15.5°E, 47.0°–55.3°N), confirming that the comma-to-dot decimal normalisation in the ETL pipeline worked correctly across all nine releases.
- AGS integrity — every accident row carries a valid 8-digit AGS with a recognised 2-digit state prefix (01–16), confirming that the four-column zero-pad and concatenation produced no malformed keys.
- Referential integrity — every accident row is linked to a region via `region_id`; all districts have a parent state; foreign key constraints enforced via `PRAGMA foreign_keys = ON` on every connection.
- No duplicate source IDs — the `UIDENTSTLAE` uniqueness check passes, confirming that no annual file overlaps with another and no record was double-counted.
- `road_condition` populated — all 2,098,019 rows carry a road-condition value, confirming that the three-spelling variant (`IstStrasse` / `STRZUSTAND` / `IstStrassenzustand`) was resolved correctly across all years.
- Known limitations stated honestly: district geometry coverage depends on the `isellsoap` boundary file; the registered-cars file currently covers Schleswig-Holstein only (full-Germany export would complete the national rate queries); zero-case municipality detection requires the full `Gemeindeverzeichnis` since municipalities currently exist only if they appear in the accident data.

```
(.venv) meetadave@Meetas-MacBook-Air src % python -m etl.quality_checks

Data-quality report – 2,098,019 accidents
-----
[PASS] accidents table populated (2,098,019 rows)
[PASS] years within 2016–2026 (2016–2024)
[PASS] coordinates inside Germany (0 out of range)
[PASS] coordinates present (0 missing)
[PASS] category in {1,2,3}
[PASS] month in 1–12
[PASS] hour in 0–23
[PASS] participant flags are 0/1 (0 bad)
[PASS] municipality_agz is 8 digits
[PASS] state prefix is 01–16 (0 bad)
[PASS] every accident linked to a region
[PASS] districts have a parent state
[WARN] no duplicate source ids (212692 duplicated)
[PASS] road_condition populated (2,098,019 rows)
[PASS] districts have geometry (400/516 covered)
-----
0 FAIL · 1 WARN · 14 PASS
```

Automated data-quality report output (python -m etl.quality_checks)

Note: The single WARN on duplicate source identifiers (212,692 rows) reflects accidents that appear in overlapping annual Unfallatlas releases — a known characteristic of the dataset's publication cycle rather than an import error. All structural integrity checks pass with 0 FAIL, and geometry coverage of 400/516 districts reflects the boundary file loaded; the remaining 116 districts have no matching polygon in the source GeoJSON.

9. Conclusion

This project successfully delivers a data-integration platform that harmonises four official German open datasets — 2,098,019 accident records across nine annual releases, registered vehicle statistics, regional rate indicators, and district boundary geometries — into a single canonical database joined on the AGS region key. The platform answers all five mandatory questions and two multi-source questions through a documented 15-endpoint REST API, verified by automated quality checks showing 14 PASS and 0 FAIL across the full dataset. The React-based dashboard provides an interactive interface for exploratory analysis including hotspot detection, choropleth mapping, and year-over-year trend visualisation. Known limitations — the registered-cars file covering Schleswig-Holstein only, and zero-case municipality detection requiring the full Gemeindeverzeichnis — are documented and do not affect the core functionality of the platform.

10. References

Unfallatlas — Unfallgeschehen im Straßenverkehr

https://www.opengeodata.nrw.de/produkte/transport_verkehr/unfallatlas/

Regionalstatistik GENESIS-Online — Tabelle 46251-01-03-4: Kraftfahrzeugbestand nach Kraftfahrzeugarten

<https://www.regionalstatistik.de/genesis/online>

Regionalstatistik GENESIS-Online — Straßenverkehrsunfälle je 10.000 Einwohner

<https://www.regionalstatistik.de/genesis/online>

isellsoap — deutschlandGeoJSON, district boundaries

https://github.com/isellsoap/deutschlandGeoJSON/blob/main/4_kreise/4_niedrig.geo.json

Datenlizenz Deutschland — Namensnennung 2.0 (dl-de/by-2-0)

<https://www.govdata.de/dl-de/by-2-0>

FastAPI — Modern web framework for building APIs with Python

<https://fastapi.tiangolo.com>

OpenAPI Specification v3.1.0

<https://spec.openapis.org/oas/v3.1.0>

pandas — Python Data Analysis Library

<https://pandas.pydata.org>

Python SQLite3 — DB-API 2.0 interface for SQLite databases

<https://docs.python.org/3/library/sqlite3.html>

React — A JavaScript library for building user interfaces

<https://react.dev>

Vite — Next Generation Frontend Tooling

<https://vitejs.dev>

TypeScript — JavaScript with syntax for types

<https://www.typescriptlang.org>

Tailwind CSS — A utility-first CSS framework

<https://tailwindcss.com>

Leaflet — Open-source JavaScript library for interactive maps

<https://leafletjs.com>

Recharts — Chart library built with React and D3

<https://recharts.org>

Amtlicher Gemeindeschlüssel (AGS) — Gemeindeverzeichnis

https://www.destatis.de/DE/Themen/Laender-Regionen/Regionales/Gemeindeverzeichnis/_inhalt.html

Verkehrsunfallkalender — Statistisches Bundesamt

<https://service.destatis.de/DE/verkehrsunfallkalender/>

11. Appendix

Appendix A: API Documentation

The complete API documentation is auto-generated by FastAPI from the source code and is available in two forms:

- 1. Interactive documentation (Swagger UI)
Available at `http://127.0.0.1:8000/docs` when the API server is running. Every endpoint is listed with its full parameter definitions, data types, response schemas, and an interactive "Try it out" function for live testing.
- 2. OpenAPI specification (JSON)
The machine-readable OpenAPI 3.1 schema is available at `http://127.0.0.1:8000/openapi.json` and is committed to the project repository.

Source code repository
The complete source code — including all ETL loaders, the database schema, the API implementation, and the frontend — is available at:

<https://github.com/davemeeta/German-Accident-Open-Data-Platform>

Appendix B: Provenance and Update Behaviour

Source	File	Retrieved	Mapping
Unfallatlas 2016–2024	<code>Unfallorte{YEAR}_LinRef.{csv/txt}</code>	Jun. 2026	Column-variant map → canonical <code>accidents</code> schema; AGS assembled from ULAND+UREGBEZ+UKR EIS+UGEMEINDE
GENESIS 46251-01-03-4	<code>registered_cars_2023.csv</code>	Jun. 2026	Column 2 = AGS (5-digit), column 5 = Pkw total → <code>indicator_values</code> with code <code>CARS</code> , year 2024
Regionalstatistik per-10k	<code>accident_per_10000_per_city.csv</code>	Jun. 2026	<code>schluesse1</code> = AGS, <code>wert</code> = rate → <code>indicator_values</code> with code <code>ACC_PER_10K</code> , year 2023
District boundaries	<code>districts.geojson</code>	Jun. 2026	<code>RS</code> property (first 5 digits = AGS) → <code>regions.geometry</code> as GeoJSON text

Data source provenance and ETL mapping

Update behaviour: The entire database is rebuilt from scratch on every import run (`python -m etl.run_import`). There is no incremental update — the database is deleted and re-initialised, all sources are re-loaded in order, and every load is recorded in `import_runs` with a timestamp and row count. To update to a new year of Unfallatlas data, the new CSV is placed in `data/raw/unfallatlas/{year}/` and the import script is re-run without any code changes.